

Design and Implementation of KARLS: A C++ KD-Tree-Based Robotic Location and Spatial Intelligence System with Applications in AI, Quantum Computing, and Blockchain

Akash Patel

Faculty Advisor: Dr. Victor Govindaswamy

Concordia University Chicago

HOME OF THE WORLD'S GREATE$T PROGRAMMERS

Abstract

- This project presents the **design and implementation of KARLS (KD-Tree Assisted Robotic Location System)**, a **C++-based application** developed to support efficient **object-location management and nearest-neighbour searching** within a simulated robotic environment. The primary goal of the system is to provide a **simple, organized, and effective solution** for managing object locations, identifying nearby objects, and calculating distances while demonstrating the practical application of **spatial data structures in Artificial Intelligence (AI) and Robotics**.
- KARLS is designed using core **Data Structures and Algorithms (DS II)** concepts, including **KD-Trees, binary tree structures, pointers, recursion, dynamic memory allocation, and Object-Oriented Programming (OOP)**. The application allows users to perform essential operations such as **adding objects, searching for objects by name, updating the robot's location, identifying the nearest object, measuring distances, displaying the KD-Tree structure, and removing stored objects**. These functionalities are organized through a **structured, menu-driven interface** to improve usability and user interaction.
- The system dynamically manages object information through **tree nodes and alternating X and Y coordinate comparisons**. **Euclidean distance calculations and nearest-neighbour searching** are implemented to identify nearby objects within the simulated environment. **Input validation** is incorporated to prevent duplicate names and locations and to handle invalid entries. **UML class diagrams** are also used to represent the application's design and major components.
- This project highlights the practical application of **spatial data structures, memory management, modular programming, and object-oriented design** in robotics. KARLS serves as both a **functional object-location simulation** and an **educational demonstration** of how fundamental computer science concepts can support **spatial searching and intelligent robotic applications**.

Technologies Used

C++ · ubuntu · Git

Future Work

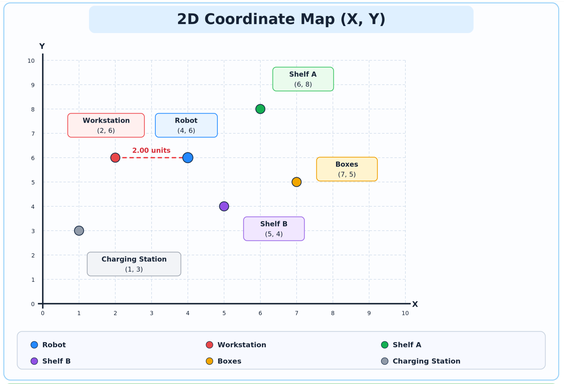
Artificial Intelligence (AI) and Machine Learning (ML): Future enhancements may incorporate **intelligent object recognition, computer vision, predictive spatial analysis, and automated robotic decision-making** to improve object identification and location-based task management.

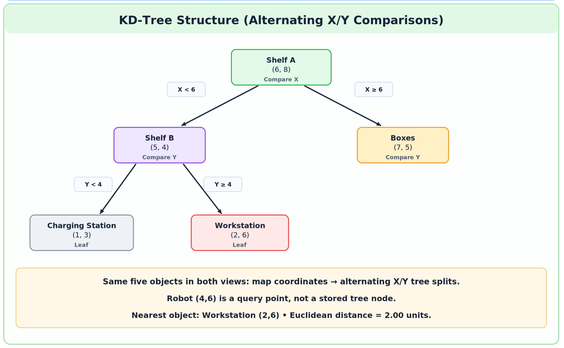
Quantum Computing: Quantum-inspired optimization and quantum search research may be investigated for **complex spatial searching, multi-robot coordination, route optimization, and robotic task scheduling**, with performance evaluated against conventional algorithms.

Blockchain Technology: Blockchain and cryptographic verification could be explored to support **secure location-data sharing, tamper-evident event histories, trusted robotic transactions, and coordination among multiple autonomous robots**.

Autonomous Robotics: Future improvements may integrate **LiDAR sensors, real-time obstacle detection, autonomous navigation, collision-free path planning, automatic KD-Tree balancing, interactive warehouse mapping, cloud synchronization, and persistent data storage** to develop a more **intelligent, secure, and scalable robotic location-management system**.

KARLS: Spatial Intelligence & KD-Tree





KARLS Main Menu

```
============================================================
  KD-TREE BASED SMART ROBOT OBJECT LOCATION SYSTEM
============================================================
1. Add Object to Robot's Environment
2. Search Object by Name
3. Set / Update Robot's Current Location
4. Find Nearest Object to Robot
5. Measure Distance from Robot to Selected Object
6. Display All Objects in Robot's Environment
7. Display KD-Tree Structure
8. Remove Object by Name
9. Exit System
============================================================
Enter your choice:
```

- This screen shows the **main menu** of the **KARLS(KD-Tree Assisted Robotic Location System).**

Option 1 — Add Object to Robot's Environment

```
============================================================
Enter your choice: 1
Enter object name: Boxes
Enter object X: 4
Enter object Y: 7
Object added successfully!
```

- Using this option, users can add objects by entering their **name and X and Y coordinates.**

Option 2 — Search Object by Name

```
============================================================
Enter your choice: 2
Enter object name: Boxes
Object found successfully!
Object name: Boxes
Object location: (4, 7)
```

- Using this option, users can **search for an object by name** and view its **X and Y coordinates.**

Option 3 — Set / Update Robot's Current Location

```
============================================================
Enter your choice: 3
Enter robot X: 6
Enter robot Y: 3
Robot location saved: (6, 3)
```

- Using this option, users can **set or update the robot's X and Y coordinates.**

Option 4 – Find Nearest Object to Robot

```
============================================================
Enter your choice: 4
Robot location: (6, 3)
Nearest object: Shelf B (8, 4)
Distance: 2.24 units
```

- Using this option, users can **find the nearest object** to the robot based on its **current location using KD-Tree searching.**

Option 5 – Measure Distance from Robot to Object

```
============================================================
Enter your choice: 5
Enter object name: Chargingstation
Robot location: (6, 3)
Selected object: Chargingstation (2, 5)
Distance from robot: 4.47 units
```

- Using this option, users can calculate **Euclidean distance** between the **robot(x1,y1)** and an **object(x2,y2)** using $\mathbf{d = \sqrt{[(x_2-x_1)^2 + (y_2-y_1)^2]}}$ **and obtain the Distance.**

Option 6 – Display All Objects

```
============================================================
Enter your choice: 6

Objects in the robot's environment (5):
1. Boxes (4, 7)
2. Shelf A (3, 5)
3. Chargingstation (2, 5)
4. Shelf B (8, 4)
5. Workstation (6, 9)
```

- Using this option, users can **view all stored objects** along with their **names and X and Y coordinates.**

Option 7 – Display KD-Tree Structure

```
============================================================
Enter your choice: 7

ROOT: Boxes (4, 7) [Compare X]
|-- L: Shelf A (3, 5) [Compare Y]
|   `-- R: Chargingstation (2, 5) [Compare X]
`-- R: Shelf B (8, 4) [Compare Y]
    `-- R: Workstation (6, 9) [Compare X]
```

- Using this option, users can **view the KD-Tree structure**, including **object locations and parent-child relationships.**

Option 8 – Remove Object by Name

```
============================================================
Enter your choice: 8
Enter object name to remove: Shelf B
Object removed successfully!
```

- Using this option, users can **remove a stored object by entering its name**, while **Option 9 allows users to exit the system.**

UML Organizational Chart

KDTree

- root : Node*
- count : int

<<constructor>> + KDTree()
<<destructor>> + ~KDTree()
+ addObject(name : const string&, x : int, y : int) : bool
+ searchObject(x : int, y : int) : const Object*
+ findNearest(x : int, y : int) : const Object*
+ removeObject(x : int, y : int) : bool
+ displayObjects() : void
+ displayTree() : void
+ getCount() : int
- insertNode(current : Node*, object : const Object&, depth : int) : Node*
- searchNode(current : const Node*, x : int, y : int, depth : int) : const Node*
- nearestNode(current : const Node*, x : int, y : int, depth : int, best : const Node*&, bestDistance : long long&) : void
- collectObjects(current : const Node*, objects : vector<Object>&) : void
- printTree(current : const Node*, prefix : const string&, lastChild : bool, side : char, depth : int) : void
- clear(current : Node*) : void

Contacts

Bioinformatics And Technology Laboratory (BAT Lab)

- **Programming Application Implementation and Design (PAID)**
- **Computer Science Department (CSC@CUC)**

Member: Akash Patel
Advisor: Dr Visvasuresh (Victor) Govindaswamy
Email: Victor.Govindaswamy@cuchicago.edu
Address: Concordia Chicago University
7400 Augusta Street
River Forest, IL, 60305-1499